Computers & Security 140 (2024) 103743



Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose





Empirical evaluation of a threat modeling language as a cybersecurity assessment tool

Sotirios Katsikeas *, Engla Rencelj Ling, Pontus Johnsson, Mathias Ekstedt

KTH Royal Institute of Technology, Division of Network and Systems Engineering, Teknikringen 33, floor 3, SE 100 44 Stockholm, Sweden

ARTICLE INFO

Keywords:
Domain-specific threat modeling language
Empirical language evaluation
Domain experts
Cyber attack simulations
Cyber security

ABSTRACT

The complexity of ICT infrastructures is continuously increasing, presenting a formidable challenge in safeguarding them against cyber attacks. In light of escalating cyber threats and limited availability of expert resources, organizations must explore more efficient approaches to assess their resilience and undertake proactive measures. Threat modeling is an effective approach for assessing the cyber resilience of ICT systems. One method is to utilize Attack Graphs, which visually represent the steps taken by adversaries during an attack. Previously, MAL (the Meta Attack Language) was proposed, which serves as a framework for developing Domain-Specific Languages (DSLs) and generating Attack Graphs for modeled infrastructures. coreLang is a MAL-based threat modeling language that utilizes such Attack Graphs to enable attack simulations and security assessments for the generic ICT domain. Developing domain-specific languages for threat modeling and attack simulations provides a powerful approach for conducting security assessments of infrastructures. However, ensuring the correctness of these modeling languages raises a separate research question. In this study we conduct an empirical experiment aiming to falsify such a domain-specific threat modeling language. The potential inability to falsify the language through our empirical testing would lead to its corroboration, strengthening our belief in its validity within the parameters of our study. The outcomes of this approach indicated that, on average, the assessments generated by attack simulations outperformed those of human experts. Additionally, both human experts and simulations exhibited significantly superior performance compared to random guessers in their assessments. While specific human experts occasionally achieved better assessments for particular questions in the experiments, the efficiency of simulation-generated assessments surpasses that of human domain experts.

1. Introduction

Protection of Information and Communications Technology (ICT) infrastructures and ensuring their security is one of the most critical challenges faced by our society today. Cybersecurity assessments play a crucial role towards securing them. The idea is simple: it will be easier to protect once we understand the security gaps (vulnerabilities) or shortcomings (weaknesses), the limits of our defenses, and what to protect against. However, as systems are becoming large and complex, this assessment is not a trivial task to undertake (Nobles, 2018). Formal methods and automated analysis tools can help experts tackle this complex problem. Indeed, for the task of assessing the security of large ICT infrastructures, threat modeling has emerged as a promising approach, and within this field attack trees and attack graphs are key concepts.

In previous works the Meta Attack Language (Johnson et al., 2018) (MAL) and coreLang (Katsikeas et al., 2020) were presented. MAL eases the cumbersome task of manually generating new attack graphs for each system, and instead provides a framework for defining threat modeling languages for different domains (e.g., industrial control systems (ICSs), vehicles and Internet of Things (IoT)) so that attack graphs can be automatically generated from system instance models. Furthermore, as many domains share a lot of common concepts and properties, it makes sense to capture these commonalities at a higher abstraction level rather than individually for all domains, before focusing on modeling the domain specifics. As its name implies, coreLang is a threat modeling language that is designed to do exactly that. It covers the basics and common structure of general purpose software systems and ICT infrastructures and as such comprises predefined assets and attacks that appear in most domains.

Developing such domain-specific languages (DSL) for threat modeling and attack simulations is a powerful means for performing security assessment of ICT infrastructures, but ensuring the correctness of such

* Corresponding author.
E-mail address: sotkat@kth.se (S. Katsikeas).

https://doi.org/10.1016/j.cose.2024.103743
Received 17 August 2023; Received in revised form 8 December 2023; Accepted 27 January 2024
Available online 1 February 2024
0167-4048/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

modeling languages is a different research question. In this study and in line with the scientific approach of Popper (2005), we conduct an empirical experiment aiming to falsify a domain-specific threat modeling language, specifically coreLang – one of our previously developed DSLs. The potential inability to falsify the language through our empirical testing would lead to its corroboration, strengthening our belief in its validity within the parameters of our study. Furthermore, another initial hypothesis we are working with, is that the simulation results will, on average, exhibit equal or greater accuracy when compared to assessments made by human experts, especially if the complexity of the analyzed system is high. If our hypothesis is confirmed, we would be content, as it would indicate that coreLang's simulation results could be effectively employed for decision support and improve the effectiveness of real-life cybersecurity assessments.

2. Related work

In a paper by Yskout et al. (2020), one of the five challenges identified for improving the maturity of the threat modeling domain is the need to define validated metrics for assessing the quality of a threat modeling process and its outcomes. The authors argue that suitable metrics to accurately evaluate if a threat model reflects reality are currently lacking.

Numerous recent empirical studies have explored the difficulty, effectiveness, and various facets of specific cyber-attacks, including ransomware attacks (Connolly et al., 2020) and zero-day attacks (Bilge and Dumitraş, 2012), as well as a human-centered approach to threat modeling that extends an existing framework and lays the groundwork for additional empirical, human-centered studies (Ferro et al., 2021). However, there is limited research concerning the empirical validation of threat modeling methods.

Considering the variety of threat modeling approaches available today, an interesting question that can be raised is which approach would be the most suitable for each modeling situation. For example, an attempt to determine which one out of the attack graphs and fault tree methods is more effective in aiding cyber attack perception is empirically evaluated in Lallie et al. (2018). While Lallie et al. (2020) provides an empirical survey of attack graphs and attack trees to identify how attack graphs and attack trees present cyber attacks in terms of their visual syntax.

A systematic literature review of threat modeling identified six different methods for validating threat models (Xiong and Lagerström, 2019). These methods include simulation, analysis, theoretical examples, empirical case studies, implementation, and expert review. The validation approach in this paper falls under the categories of simulation and expert review.

In a paper identified in the literature review by Xiong and Lagerström (2019), an expert review was conducted. The authors used a threat model to identify threats that would indicate to an attacker the presence of a honeypot (Dahbul et al., 2017). They enhanced the honeypots to make them more difficult to detect and asked two professional penetration testers to identify the honeypots without any prior information about the hosts. The authors compared their threat model against the experts' findings. This approach of comparing experts' assessments with the threat model is similar to the one presented in this paper. However, our study involves multiple experiments and gradually providing information to the experts to further evaluate the threat model.

Similarly, in a study by Allodi et al. (2022), a novel and realistic attacker model was presented and validated using a quantitative empirical approach with a large dataset of attacks against over one million real systems. However, a limited number of works focusing on the empirical validation of threat modeling languages were found through a literature search.

Furthermore, a systematic method for developing and evaluating MAL-based domain-specific languages was proposed in Hacks et al. (2022), where a survey revealed that only 9% of the developed MAL-based DSLs had been evaluated by industry/security experts at that time.

Regarding the validation of threat modeling languages specifically, an example is the use of the Feigenbaum test (Katsikeas et al., 2022). The Feigenbaum test (Feigenbaum, 2003) is a type of Turing test in which a human judge determines if a behavior is caused by another human or a computer, with the underlying objective being that the computer aims to perform as well as a human, thereby demonstrating human-like intelligence and behavior. In this paper, the evaluation involves both experts and a computer simulation, but we do not employ a human judge to assess the results. Instead, we consider the penetration testing training data we have gathered as the reference value for evaluation. To assess the simulation results and expert opinions against the penetration testing training data, we utilize a method inspired by the Analytic Hierarchy Process (AHP) (Saaty, 1984). AHP allows for the mathematical evaluation of the complex problem through pairwise comparisons. In our study, we adopt a similar scale, as proposed on the AHP article, but we instead focus on assessing the difficulty of achieving different hacking goals.

3. Background

3.1. The cyber range

When evaluating the domain-specific threat modeling language, we compare the assessment results to what we consider to be the ground truth. That ground truth originates from the penetration testing training data that was collected from a capture the flag (CTF) exercise on a virtual cyber range as part of an academic course at university level. In order to produce comparable results, the exact same cyber range is used by the human experts in order to evaluate the difficulty of the different attack scenarios, and for the attack simulation's assessment a model of that cyber range is created. The network topology of the deployed cyber range is shown in Fig. 1.

The cyber range is deployed in Google Cloud and consists of 79 virtual machine instances that are created and managed with Ansible. The students have access to the cyber range via a VPN connection. The experts, on top of that, will additionally have access to the Google Cloud management console. Through the console they will be able to directly inspect the configuration and get more information regarding the infrastructure.

In the cyber range, a number of previously detected cybersecurity vulnerabilities and weaknesses are deployed. The flags are text strings that have been placed throughout the cyber range. These can only be found when the exploitation of a vulnerability and/or weakness has been successful. The flags have been named with sequential numbers but those numbers do not necessarily follow the order with which they are found inside the cyber range. In Table 1, a detailed list, of all the flags and their corresponding topics can be found. One of the instances is the VPN server and the 17 flags are contained in six other instances. The rest of the instances are honeypots and do not contain any flags. The instances are separated into two networks. One network is the external (i.e. it is accessible from the VPN connection), and behind the firewall is the internal network which is not accessible from the VPN connection. On the external network, a honeypot acting as a router is also deployed. For the six instances with flags, 3 are before the firewall (Instance #1–3) and the other 3 are behind it (Instance #4–6), as shown in Fig. 2.

3.2. The Meta Attack Language (MAL)

The Meta Attack Language (MAL) (Johnson et al., 2018), that this study is based on, was designed to enable automatic generation of attack graphs from a system architecture model. The purpose of this was to enable security assessments of system architectures. MAL can be used to define domain-specific languages for different domains, such as industrial control systems (ICSs), internet of things (IoTs), vehicles,

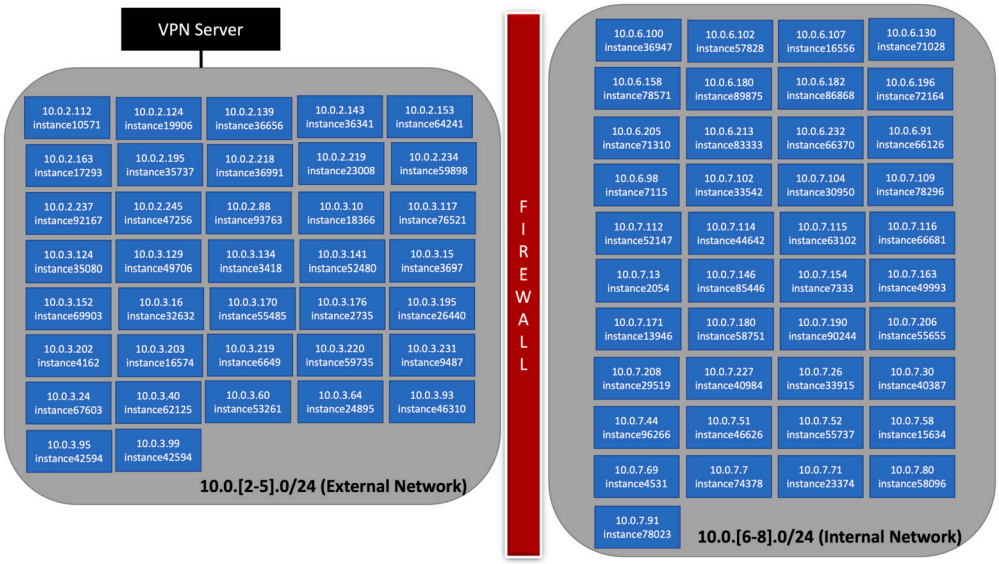


Fig. 1. The deployed cyber range network topology.

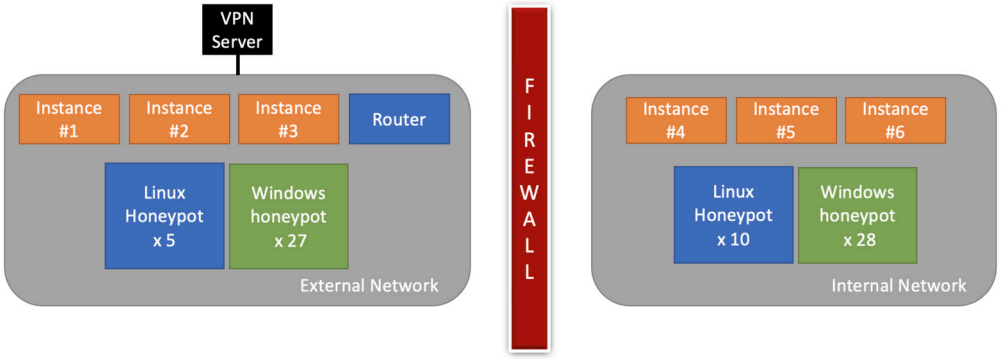


Fig. 2. Simplified view of the cyber range network topology.

Table 1
Flags and their topics.

Flag	Topic
Flag 1	Password cracking
Flag 2	Web hacking/Password cracking
Flag 3	Remote exploitation
Flag 4	Hash cracking/lateral movement
Flag 5	Privilege escalation
Flag 6	Web crawling
Flag 7	Database hacking
Flag 8	Web hacking
Flag 9	Privilege escalation
Flag 10	Traffic sniffing
Flag 11	Client-side attack
Flag 12	Binary exploitation
Flag 13	Remote exploitation
Flag 14	Cloud hacking
Flag 15	Cloud hacking
Flag 16	Wifi cracking/Traffic sniffing
Flag 17	Wifi cracking

etc. Conceptually, a MAL specification of a system consists of different `assets` (e.g., Host), `attack steps` on the assets (e.g., Host.Compromise), `defenses` on assets (e.g., Host.EndPointDetection) and finally `associations` between different assets. Additionally, to encode attack graph logic, attack steps are connected to each other, which means that the successful compromise of one attack step enables attempts at other attack steps. Attack steps can be of two types, OR or AND, respectively indicating that compromise of any individual parental attack step is required (OR) or compromise of all parental attack steps is required (AND) for the current step of the attack to be performed. Moreover, defenses are related to attack steps, and if enabled, act as obstacles to the connected attack steps. Additionally, a probability distribution, called the local Time-To-Compromise (TTC), can optionally be associated to each attack step, representing the difficulty of succeeding with that attack step. The global TTC, on the other hand, represents the difficulty, expressed in time, required for an attacker to succeed with a complete attack. Defense steps are probability distributions over Boolean values representing if they are active or not.

MAL-based languages are used to generate attack graphs that describe the possible attack paths to compromise the assets that make up a model, instantiated in the language. Those attack paths are presented as a graph where each node of the graph is an attack step required for continued progress along the path.

In Fig. 3, the basic modeling MAL pipeline's four steps are depicted. It begins with a given MAL-based language from which the asset instance model is derived. This leads to the generation of the complete attack graph, representing all possible attack paths within the modeled infrastructure. The graph can then be pruned to reveal the shortest path to a specific attack goal.

3.3. coreLang

Many ICT infrastructure environments share several components, each displaying similar behaviors and properties. This has motivated us to develop a language based on MAL, which encapsulates these commonalities. coreLang provides a collection of predefined assets that offer

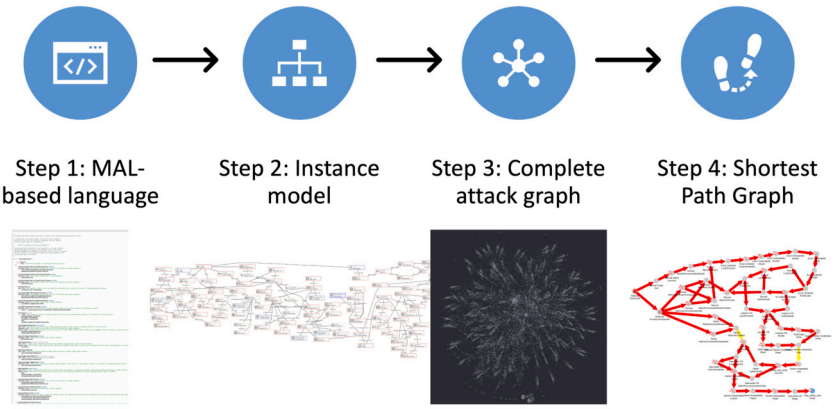


Fig. 3. The modeling MAL pipeline (the figures below each step should not be legible and are there for showcasing the format of each output).

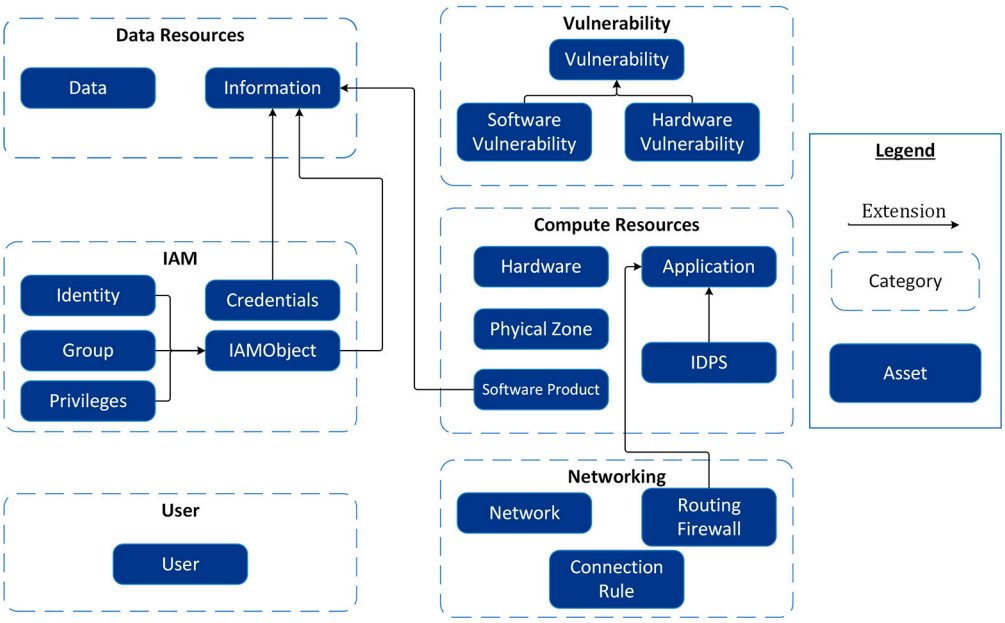


Fig. 4. Overview of coreLang.

a basic modeling framework for that domain. Thus, it can serve both as a starting point for the creation of more domain-specific languages but can also be used to model generic environments. For a detailed background and motivation behind coreLang, the interested reader is referred to Katsikeas et al. (2020) and on the GitHub repository of the language.[1]

Modeling generally entails a level of abstraction as it seeks to capture the relevant interactions between elements while discarding the extraneous aspects. Due to coreLang's generalist scope, the language operates at a relatively high abstraction level. The primary goal is to accurately depict the essential behavior of components commonly found in ICT infrastructures. This means that the assets of the language have to be generic and therefore do not capture specialized behavior regarding their functionality. For example, a database manager and a video processing app will look the same from coreLang's perspective, both as an `Application` that has some `Data` associated with it. The interactions between assets and the enabling of particular defenses, can however, often capture the most dominant aspects relating to the specific purpose of an asset. A modeler could, for instance, specify that the database `Application` uses multiple `Data` assets as backup storage, thus making it more resilient against an attacker's disruption attacks.

coreLang has been under development for the past three years with the goal of encompassing existing cyber attacks on ICT infrastructures. The language has undergone iterative validation through a series of brainstorming sessions with practitioners. Recently, it has also been validated against the MITRE ATT&CK Enterprise Matrix (MITRE, 2024a), which is currently considered a kind of de facto standard in the community as a comprehensive knowledgebase of known attack vectors, techniques, impacts, and mitigations (Tayouri et al., 2023).

However, it is important to note that the majority of the probability distributions and their assigned values on the attack steps within coreLang have not yet been thoroughly researched. They are based on our assumptions and estimations.

Fig. 4 presents the overall structure of coreLang. Assets in coreLang are grouped into six categories: `Compute Resources`, `Vulnerability`, `Data Resources`, `User`, `IAM (Identity and Access Management)`, and `Networking`.

3.4. Performing cyber attack simulations

To perform attack simulations using coreLang as the MAL-based domain-specific modeling language, the securiCAD Professional tool provided by foreseeti (Ekstedt et al., 2015) was used. The simulation,

[1] The first release version of coreLang can be found under: https://github.com/mal-lang/coreLang. The release version is an improved iteration of the language presented in Katsikeas et al. (2020).

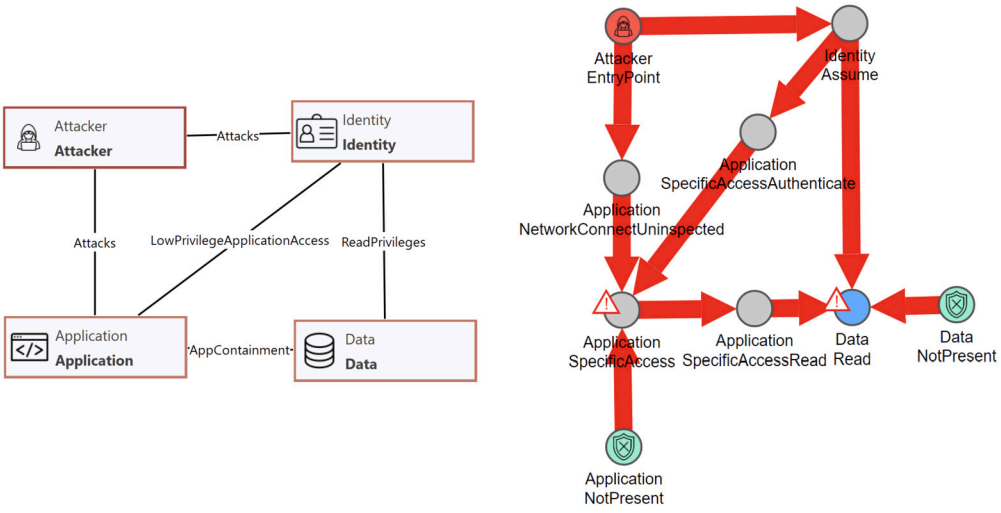


Fig. 5. Accessing Data hosted on Application using an assumed Identity in coreLang.

performed with MAL and securiCAD, is based on probabilistic simulation in attack graphs, similar to Bayesian networks, where a shortest path algorithm is used to identify the possible attack path(s) towards each attack target. This approach is widely used today for protecting some of the largest infrastructures in the world (Nydrén, 2023).

Once the simulations have been performed, we analyzed the maximum success rate for each attack goal. That maximum success rate is calculated using the powerful computational technique known as the Monte Carlo method. This method involves performing multiple simulations with random sampling to estimate the maximum success rate while considering a given (global) TTC threshold as the upper bound. This TTC threshold is equivalent to the attacker time budget which is the maximum time the attacker can spend attempting a specific attack goal. Therefore, the use of the word maximum as a prefix in the success rate refers to the point in time when the attacker's time budget is depleted.

In Fig. 5, an example instance model, created using coreLang, of an attack originating from an assumed *Identity* is depicted on the left side. While on the right side of the same figure, the attack path graph that is generated from the instance model, and which contains all the involved attack steps, is depicted. Then, in Listing 1, a code snippet of all the involved attack steps from coreLang is presented. In this model, the simulated *Attacker* can be seen on the top, and is associated with both the *Identity* instance that it assumes/compromises but also with the *Application* instance on which it achieves a network connect, which is the equivalent of being able to reach that application by a network. Then, by looking on the attack path graph, we can see that the *Attacker* can also read the data associated with the assumed *Identity* which are contained in the associated *Application*.

For reference, the largest model created for this study, contained 233 asset instances and those were connected with each other with 273 relations.

4. Methodology

To empirically evaluate coreLang, we designed and conducted a series of experiments where we asked different types of assessors about the difficulty of performing specific cybersecurity attacks on the cyber range. These assessors were: i) human experts in the cybersecurity domain, ii) cyber attack simulations performed by coreLang, and iii) random guessers. We then compared the assessments from our various types of assessors with the penetration testing training data we had collected about the cyber range, which constitutes the ground truth. In those experiments, all assessors were asked to answer the same set of pairwise comparison questions regarding which attack goal is eas-

```
asset Identity extends IAMObject{
      | assume
        -> execPrivApps.authenticate,
           highPrivApps.authenticate,
           lowPrivApps.specificAccessAuthenticate,
           ...
           managedIAMs.attemptAssume,
           subprivileges.attemptAssume
      ...}

asset Application{
      | networkConnectUninspected
        -> attemptUseVulnerability,
           ...
           networkConnect,
           specificAccessNetworkConnect

      | specificAccessNetworkConnect @hidden
        -> specificAccessFromNetworkConnection

      & specificAccessFromNetworkConnection @hidden
        -> specificAccess

      | specificAccessAuthenticate
        -> specificAccessFromLocalConnection,
           ...
           specificAccessFromNetworkConnection

      | specificAccess
        -> appExecutedApps.localConnect,
           specificAccessRead,
           ...
           bypassContainerization,
           attemptUseVulnerability,
           accessNetworkAndConnections

      | specificAccessRead {C}
        -> containedData.authorizedReadFromApplication,
           sentData.authorizedReadFromApplication,
           receivedData.authorizedReadFromApplication
      ...}

asset Data{
      | authorizedReadFromApplication @hidden
        -> read

      | read {C}
        -> containedData.attemptRead,
           information.read,
           extract
      ...}
```

Listing 1: coreLang code snippet for the attack steps of Fig. 4.

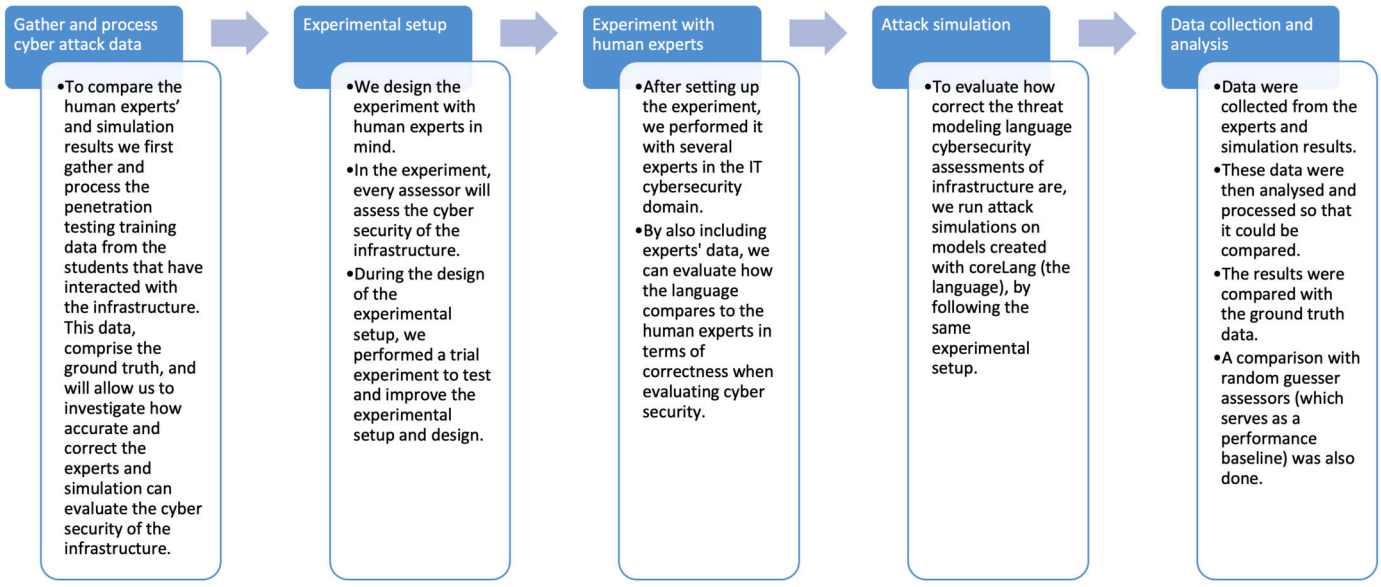


Fig. 6. Overview of the methodology for our study.

ier for a professional penetration to succeed with. In Subsection 4.1, we give details of how we collected and analyzed the penetration testing training data about the cyber range to construct the ground truth measures of relative attack path difficulty on which all the comparisons are based. In Subsection 4.2, we describe the experimental setup for conducting this experiment. Then, in Subsection 4.3, we present the methods used to collect and analyze the data provided by our assessors. In Subsection 4.4, the use of random guessers as a performance benchmark is presented. Next, in Subsection 4.5, we discuss the human domain expert participants and how we analyzed their results to be able to compare them to the ground truth. We also present the pilot run experiments that were performed before the actual experiment. Lastly, in the Subsection 4.6 we describe the method of how we perform the attack simulations, how we collect the simulation's assessment data and how we analyze them to create comparable results. Fig. 6 presents an overview of the methodological approach used in our study.

4.1. Ground truth measures of relative attack path difficulty

In this section, we will present the process by which we collected and constructed the ground truth data of relative attack path difficulty in our analysis.

The penetration testing training data from our cyber range is derived from three distinct sources: i) `Students A` – 225 master's level students at KTH Royal Institute of Technology who undertook the course in the autumn of 2021 and performed all hacks necessary to capture all flags and pass the course; ii) `Students B` – 38 students who enrolled in a variant of the course during the same period (though the cyber range and assignments remained identical) and provided feedback on the perceived difficulty of each hack/flag, by selecting among five different difficulty levels in an ordinal scale, as well as the duration taken to capture each flag; iii) Our own assessments of flag difficulty, as we are the cyber range maintainers and course instructors.

The collected data from `Students A` contain the grades that each student got in the course for each flag. For each flag, a number of on-demand hints are available, which offer guidance to assist in finding and capturing the respective flag. The grade a student will get for each captured flag will be reduced in the case the student requests hints for that specific flag. By inverting the grade of each student for each flag, we were able to calculate the hint points required to capture that flag. Then, by calculating the average hint points needed for each of the 17 flags, we established a measure of difficulty, represented by this average hint point value. Based on our experience with the course, some of the students request hints for flags due to time constraints or indifference to the grade rather than difficulty, we have opted to include data only from the top 80% of `Students A` in our calculations in order to counteract this.

To compute the difference in the number of required hint points between any two targets A and B, we considered any intermediate steps needed, utilizing our existing knowledge of the cyber range. This was achieved by simply summing up the average hint points needed for each target and calculating the difference. If for example target A is the first one and requires in average 1 hint point, target C is another target that requires in average 4 hint points, and target B is the intermediate flag that requires in average 2 hint points and has to be captured before reaching target C, then the hint points needed for target C would be 1+2+4 = 7 and the difference from target A to target C would be 1-7 = -6. We already know how the flags are arranged and the intermediate steps required for each from the cyber range.

To determine the relative ease or difficulty of achieving an attack goal A compared to another attack goal B, we utilize a numerical value from our own adapted version of Saaty's 1–9 scale (Saaty, 1984), with values in the range [0,8] where a value of 0 represents that both attack goals are equally easy to achieve, and a value of 8 represents that attack goal A is exceptionally easier than attack goal B. We call that Scale of Ease.

By applying the process described above to all pairwise comparisons and after normalizing the results to the [0,8] range, we can represent the relative difficulties on our Scale of Ease.

From the second data source, namely the difficulty feedback provided for the flags by `Students B`, we can adopt a similar approach. However, this time we apply it to the difficulty scores instead of the hint points needed for each flag. This was facilitated by assigning a numerical value to each of the difficulty scores available (very easy, easy, moderately easy/difficult, difficult and impossible) and then calculating the average of the student responses for each flag. Consequently, we can calculate the differential difficulty for all of our pairwise comparisons, with the same approach as for the hint points using the necessary intermediate steps to reach a flag, and once again, normalize these values to the [0,8] range.

Finally, for our own assessments for the relative difficulty of each flag, we can directly provide our estimated difficulty scores, and given again the intermediate steps required to reach a flag, calculate the answer values for the pairwise questions.

Table 2
Summarized experimental setup for all assessors.

Experiment	Assessment size	[Level] of information provided to human experts/added in the model	Time Available	Number of Questions
Experiment 1	Single host	[Low] Only Nmap of the host (i.e. identified network services) The experts will have to provide their best guess	30 Minutes	5
Experiment 2	Half cyber range (before firewall)	[Moderate] Cyber range network topology (diagram) Nmap scan of 37 hosts Lengthy Nessus scan reports for 3 hosts that contain flags	45 Minutes	9
Experiment 3	Half cyber range (before firewall)	[Full] List of vulnerabilities (Nessus scan reports + manually complemented by us if something was missing) Credentials metadata (and their location in cyber range) Cyber range network topology (diagram) Nmap scan of 37 hosts SSH credentials for the hosts containing flags	45 Minutes	9
Experiment 4	Full cyber range	[Full] Same as above but for the whole cyber range: Lengthy Nessus scan reports for 5 hosts that contain flags Nmap scan of 78 hosts	5 Hours	24

Each of these three data sources represents the same attribute: the difficulty level of each attack goal/flag. However, due to their varying ranges, we standardized the data by calculating the differential difficulty and then normalizing it to our Scale of Ease. Consequently, all the data were processed using a consistent or very similar approach.

What the truth *really is*, is clearly a question without a single objective answer. For this study we decided to create the ground truth by merging the three data sets into a single ground truth by selecting the median Scale of Ease value among those three groups assessments as we found no strong arguments for considering one data set *more true* than another.

4.2. Experimental session setup

The experimental session has been designed to maximize the available comparison points by considering three key dimensions. These dimensions are: i) the size of the network infrastructure that the assessors interact with at each time, ii) the time that the assessors have available to analyze the network infrastructure, and iii) the amount/level of information that was shared with the assessors before their assessment.

The experiment is divided up into four sub-experiments, namely Experiment 1–4, where the three dimensions (size of network infrastructure, available time, and level of information) are varied. This is done because we aim to study how those three dimensions affect the accuracy of each assessor's assessments.

In the experimental session, we requested each assessor to answer numerous pairwise comparison questions. These queries were centered on whether specific targets or attack objectives in the network infrastructure were easier to reach or achieve in comparison to others. They should consider those questions for an attacker that is a professional penetration tester and only has access to the cyber range via the VPN connection.

Our aim was to obtain responses from all assessors to the posed questions, enabling us to compare their answers. Furthermore, as described in Subsection 4.1, we now possess knowledge about the correct answers to the questions. Consequently, we have prior knowledge regarding the feasibility and relative difficulty of achieving specific attack goals compared to others. Additionally, during the experimental design phase, we had not yet created the coreLang model of the infrastructure which means that the accuracy of simulation results for the specific attack goals in question was at that point unknown to us.

To determine the relative ease or difficulty of achieving an attack goal A compared to another attack goal B, the assessors utilize the previously presented Scale of Ease.

In the subsections following, how this is achieved for all types of assessors will be described in detail.

To ensure a fair comparison, the same level and type of information is consistently provided to both the human expert and the attack simulation assessor. Additionally, the experiments were designed with the requirement to be completed within a single day to accommodate the participation of the human expert assessors. As already mentioned, the three dimensions have been used in four different experiments and details about how each dimension was used in each one of the four experiments is summarized in Table 2. Then, adjusting the designed experiments to the human experts and to the attack simulation assessors would be a straightforward task to perform and is described in Subsection 4.5 and Subsection 4.6, respectively.

4.3. Method for collecting and analyzing the assessors' data

In this study, data from different types of assessors were collected through their answers to pairwise comparison questions regarding the ease of achieving specific attack goals for a professional penetration tester. With a total of 17 attack goals available in the cyber range, the number of unique pairwise comparison questions that could be asked for k elements is given by the formula: k(k-1)/2. This results in a total of 136 pairwise comparisons. Asking all 136 questions to the human experts would be impractical, so we selected a subset of 5 pairwise comparison questions for Experiment 1, 9 pairwise comparison questions for Experiments 2 and 3, and 24 pairwise comparison questions for the longest Experiment 4, considering the time available for the assessors to avoid overwhelming them with too many questions. The question selection process followed a mixed approach, considering comparisons that made sense for the experts (e.g., comparing attack goals with varying difficulty) and ensuring that each of the included attack goals, in each experiment, had at least one question.

All the collected data from the assessors are stored as answers to the pairwise comparison questions. For example, a data point could be represented as {flag A, flag B} = 4, indicating that flag A is easier compared to flag B with a score of 4 on our Scale of Ease. Another example would be {flag C, flag D} = -7, implying that flag D is easier than flag C with a value of 7.

In the analysis and comparison of the collected data, which are in the format of the Scale of Ease and range from 0 to 8, we directly compared the answer values. To assess correctness, we employed the root mean square error (RMSE) as a measure. RMSE is widely used to evaluate the accuracy of estimators or assessors and is preferred over mean square error (MSE) because it provides an error metric on the same

scale as the original variable, making it easier to interpret the average prediction error.

4.4. Random guesser benchmark

To gain further insights and provide a meaningful comparison, we also included a random guesser in our study, following the approach described in Schmidhuber et al. (2001). The purpose of comparing against random guessers was to evaluate whether the human experts and the simulation results outperformed random chance or not. The random guesser in our study was implemented as a pseudo-random number generator. It generated 50 different random sets of answers to the same set of questions that were given to the other assessors. For each random guess iteration, the root mean square error (RMSE) was calculated by comparing the random guesses against the ground truth. The average RMSE of the random guesser was then computed and compared with the RMSE values of the human experts and the simulation results.

4.5. Human experts assessments

The complete list of questions that were asked to all of the assessors, including the human experts, is available at.[2] An example of one of the questions asked is presented below:
Which of the following is easier:

a. Actively interact with and, if necessary, successfully authenticate to SSH service at host Malval-instance78296 (10.0.7.109)
b. Obtain from within the cyber range, by using any method, the credentials for the SSH service at host Malval-instance23374 (10.0.7.71)
c. They are equally easy (0 on the scale)

If you chose a or b, according to the Scale of Ease, how much easier is the option that you selected?

During the experimental session, the human experts used their own computers to assess the security, and we conducted the experiment over a video meeting. This setup allowed the experts to ask any questions to us during the experiment, and we were able to assist them in setting up the environment before they began answering the questions. In cases where an expert was not familiar with GCP, we provided a brief introduction to the interface, guiding them on where to find the necessary information before proceeding with the experiment.

Additionally, in the case of the human experts, we asked them to assume the role of system architects. As part of this role, we provided them with access to the Google Cloud Platform (GCP) to assess the security of the system. By using the GCP platform, the experts were able to gather information such as host names, host types, metadata, and configuration details (such as executed services, IP addresses, open ports, etc.) of the hosts in the cyber range. This access could enable them to analyze the security of the system more effectively.

4.5.1. Human experts experimental setup

The details on how the experiments were adjusted to the human expert assessors are presented below and are also summarized in Table 2.

- Experiment 1: low information, short time and single host
  In the first experiment, the human experts were provided with the results of a network scanning tool, specifically the *nmap* software. This was done to save time for the experts, as they did not need to perform the scans themselves. In this experiment, the experts focused their analysis on one specific host within the cyber range. The experts were given a limited timeframe of 30 minutes to provide their best guesses (due to the absence of advanced reconnaissance) in response to the 5 questions posed to them. Since only one host is of interest on this experiment, some of the questions asked are about unreachable attack goals. It is important to note that in the subsequent experiments, the experts' visibility over the cyber range was expanded.
- Experiment 2: medium information, moderate time, medium size range
  In the second experiment, the human experts are given a longer duration and more information to analyze. Similar to the first experiment, they receive the results of a network scanner (specifically, *nmap*). Additionally, they are provided with the network topology of the infrastructure and the results of a vulnerability scanner, in this case, the basic network scan functionality of the *Nessus* scanner. Furthermore, the experts are granted VPN credentials to access the cyber range, allowing them to collect any additional information they may need. The scope of analysis is expanded in this experiment, as the experts are now tasked with assessing all instances in the external network of the cyber range. They are asked to provide answers to nine questions. The time allotted for this experiment is extended to 45 minutes to accommodate the increased complexity and amount of information. They have to answer 9 questions, which are all related to different reachable attack goals in the cyber range.
- Experiment 3: full information, moderate time, medium size range
  In the third experiment, the human experts are provided with all available information to assess the security of the cyber range. This includes detailed information about the network topology, connections between hosts and services, assigned privileges, and the presence of vulnerabilities and weaknesses on each service. The experts receive this information in the form of a supplemental list of vulnerabilities and weaknesses, which includes the corresponding Common Vulnerabilities and Exposures (CVE) (MITRE, 2024b) and Common Weakness Enumeration (CWE) (MITRE, 2024c) IDs for each instance. They also receive SSH credentials for accessing all the instances located in the external network of the cyber range via their SSH backdoor services.
  Additionally, the experts have access to a list of metadata for all the credentials found in the cyber range. The metadata includes details about services with credential-based authentication, specifically highlighting whether a default or easily guessable username is used and information regarding the complexity and strength of its corresponding password. This information is provided to them in addition to any previous information they have received from us or gathered themselves during the previous experiments. From this experiment onwards, the experts did also utilize the GCP console to assist in their assessment.
  It is important to note that the same detailed information is also present in the coreLang model for the cyber range in the full information scenario, ensuring consistency between the information provided to the experts and the simulation model. The experts are asked to provide answers to the same set of questions as in the previous experiment, and they are given the same amount of time to complete their assessments.
  The time allotted for this experiment is once more 45 minutes and the experts have to answer the same questions as on Experiment 2.
- Experiment 4: full information, longer time, entire range
  In the fourth and final experiment, the human experts are provided with all available information, as on Experiment 3. They are given a significantly longer time of 5 hours to analyze the entire cyber range and provide their assessments. The experts are asked to answer 24 questions, which are related to different attack goals in the cyber range. Among these attack goals, 80% of them are realistically achievable, while the remaining 20% are intentionally asked about the honeypots or not existing attack goals. By including these unreachable attack goals, the experiment aims to evaluate

[2] https://github.com/KTH-SSAS/coreLang-validation.

the ability of the experts to identify and differentiate them from the legitimate attack goals.

4.5.2. Trial experiments

Before conducting the full-scale experiment with our selected human experts, we conducted a pilot run consisting of two smaller-scale and shorter trial experiments. The purpose of these pilot experiments was to verify the feasibility of our experiment goals and ensure the experimental design was sound. Two non-expert participants, who were cybersecurity PhD students at KTH Royal Institute of Technology, were invited to participate in the pilot run.

During the pilot run, we focused on assessing the effectiveness of accessing the Google Cloud Platform (GCP) interface and utilizing VPN access to gather information. We provided a brief introduction to the GCP interface, demonstrating its use in managing a GCP infrastructure and obtaining relevant information.

Due to time constraints, we only ran Experiment 1 and Experiment 4 in the pilot run. This allowed us to make adjustments and improvements based on the results. For instance, we realized the need to reduce the number of questions for shorter experiments and provide more information to the experts in Experiments 2 to 4. We also recognized the importance of providing a detailed list of all available sources of information for each experiment, ensuring experts have access to all relevant data and minimizing the risk of overlooking important details.

4.5.3. The human domain experts

In our experiments, a total of twelve human experts from the cybersecurity domain participated. Nine of these individuals originated from the private sector, whereas the remaining three were from academia. The expert practitioners occupied various roles, extending from seasoned cybersecurity consultants to principal cybersecurity advisors and business security officers at prominent companies. The experts from academia consisted of two postdoctoral students and one doctoral student at KTH Royal Institute of Technology and Stockholm University, respectively. Within the last four years, none of the experts had taken the university-level course which utilizes the cyber range used in this study.

We conducted twelve individual experiment sessions with the experts. Each session had a maximum duration of eight hours, with a total of seven hours allocated for the experiments themselves. The remaining one hour was reserved for setting up the connection to the cyber range, providing a brief walkthrough of the GCP console, and addressing any questions they had before commencing the experiments. We selected this time limit to strike a balance between obtaining a thorough analysis from the human experts and accommodating their maximum availability within a single day.

During the expert selection process, we did not conduct tests to identify the most skilled individuals among the ones that responded to our time-bounded open call for participation; instead, we relied on basic background checks, performed by us, designed to verify their competence to undertake similar professional tasks. By doing that, we could regard them as a benchmark (representing the average cybersecurity expert), since they were randomly selected individuals with comparable experiences and professions. Additional details about our human experts can be found in Table 3.

After gathering the pairwise comparison answers from the experts, we calculated the root squared error for each of their individual answers. This allowed us to measure the accuracy of each expert's assessments compared to the ground truth. We also computed the average RMSE across all the experts, providing an overall performance measure.

Table 3
Facts about our human cybersecurity domain expert participants.

	Cybersecurity experience	Current title
Expert 1	2 years	Postdoctoral researcher in cybersecurity
Expert 2	5 years	Senior information security architect
Expert 3	3 years	Cyber security specialist
Expert 4	3 years	Doctoral student in cybersecurity
Expert 5	10 years	Postdoctoral researcher in cybersecurity
Expert 6	8 years	Independent cybersecurity consultant
Expert 7	10 years	Head of BA research & development
Expert 8	30 years	Trusted principal cyber security advisor
Expert 9	7 years	Risk and Threat manager – IP networks
Expert 10	9 years	Senior cybersecurity software engineer
Expert 11	14 years	Solution architect and ethical hacker
Expert 12	19 years	Cybersecurity consultant

4.6. Attack simulation assessments

The threat modeling security analysis of the cyber range is based on the MAL-based language coreLang. The aim of this paper is to empirically evaluate the MAL-generated security assessments, which is why we perform several automatic attack simulations. These simulations are performed on models of the same infrastructure that the human experts interact with, and these models have been created using coreLang. It is important to note that the human expert assessors were provided with the same level and type of information that exists in the models of the cyber range on which the attack simulations are conducted. Furthermore, the same attack goals, including the unreachable ones, for which the human experts were asked to provide assessments were also selected from the simulation results for direct comparison and analysis.

In the list below, we outline the different simulation models used for each of the experiments performed. Moreover, it must be noted that the creation of the coreLang-based models was conducted by two authors of this study, one of whom had no involvement in coreLang's development.

In general, it was expected that models with less comprehensive information, such as less detailed representations of the infrastructure, would make it harder for the simulation to provide accurate and realistic results.

- Experiment 1: Low information model with only host Instance #1
  For this scenario, the human experts were only given the *nmap* scan results without any additional vulnerability information. Consequently, we also provide the coreLang model with a list of services being executed on each instance. The model was, therefore, not aware of any vulnerabilities, credentials, or lack of defenses. As a result, we assumed that the simulation generated by this model will not be very realistic and may not be able to produce conclusive results.
- Experiment 2: Moderate information model with the hosts before the firewall
  In this model, the simulated attacker does not automatically have access to the credentials that are easily guessable, as in the full information version. Instead, the credentials were given probabilities for being disclosed, with a value of 0.5 for FTP credentials and 0.9 for web server credentials. The values for the probabilities were chosen as rough estimates to represent the ease of guessing the credentials. For instance, the password for the web server was set as the default one, making it relatively easy to guess, while the password for the FTP server was short and easily guessable.
  In this model, software vulnerabilities were added on all the probably vulnerable network services. For the software vulnerability on the host with the web server service, the probability that the vulnerability does not exist was set to 0.5. For all the other vulnerabilities, this probability was set to 0.1. Again, the chosen values were meant to reflect the uncertainty regarding the existence of vulnerabilities. For the vulnerability with the highest probability of existence but lacking specific information, a value of 0.5 was set as an average estimate. On the other hand, all other vulnerabilities were assigned a probability of 0.1, indicating higher uncertainty about their existence.

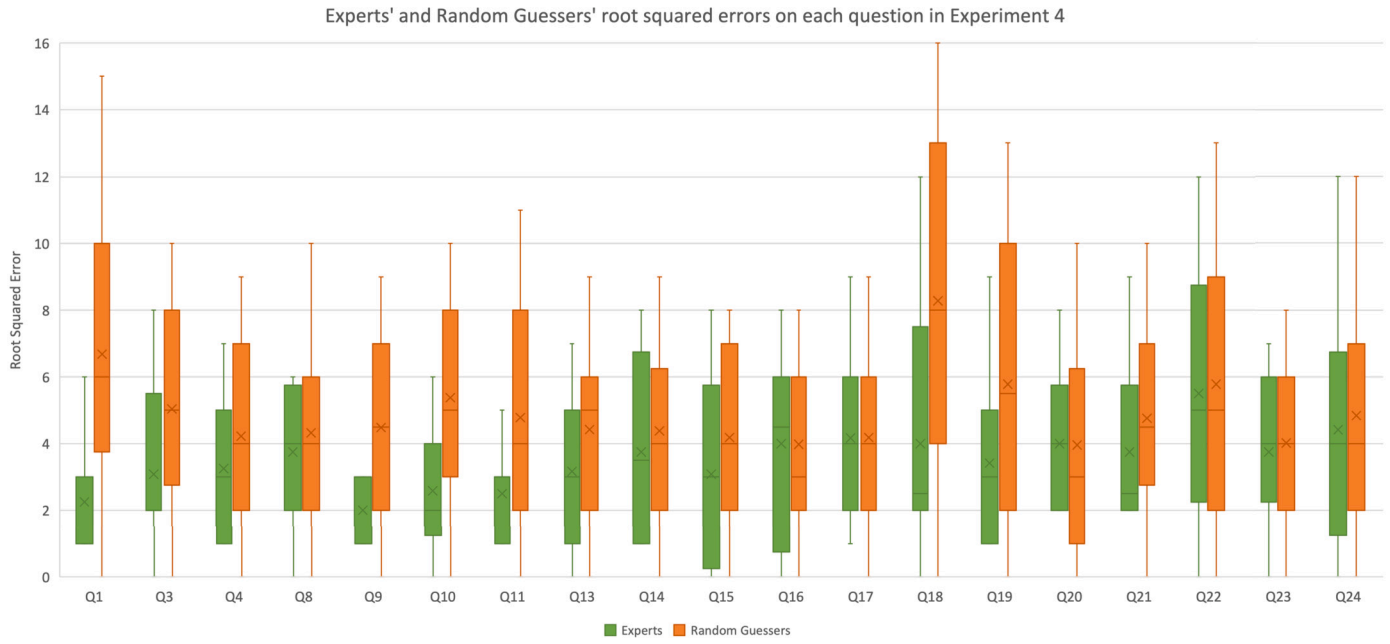


Fig. 7. Experts' and Random Guessers' root squared errors per question in Experiment 4.

- Experiments 3 & 4: Full information model covering half/entire cyber range, respectively
  In this model, SSH backdoor services have been added to instances Instance #4 and Instance #6. The model aims to provide a detailed representation of the cyber range, consistent with the information that the human expert assessors receive in Experiments 3 and 4.

The details on what level and type of information were provided to the models are also summarized in Table 2.

As already presented in Subsection 3.4, one of the results of the simulations is the success rate for each attack goal. In our study, we conducted 1,500 Monte Carlo simulations and used a uniform distribution for the TTC threshold, ranging from 0 to 50 days. The choice of 50 days as the upper bound was based on the maximum amount of time available to the `Students A` group from the penetration testing training data for capturing all the flags. We sampled this distribution multiple times and recorded the success rate for each attack goal at each sampled TTC threshold.

By comparing the success rates of different attack goals, we can determine the relative difficulty between two targets. The differential success rate, which represents the difference in success rates between two targets, is used as a measure of their relative difficulty. For example, if target A has a success rate of 100% and target B has a success rate of 65%, we can conclude that target A is easier with a differential success rate of 35%. To calculate the answers to the pairwise questions for the attack simulations, we normalize these success rate differences to a scale of 0 to 8.

Same as with the human expert data, the RMSE of the simulation data compared to the ground truth is calculated.

5. Results

In this section, we will present the results of our study, while a more detailed analysis of these results will be provided in the next section.

On average, our experts took five hours to complete all the experiments, with the longest session lasting the same duration. The average and longest time spent by the human experts as well as the time required by the simulation per experiment is detailed in Table 4.

Table 5 presents the average RMSE for all experiments, comparing the human domain experts' assessments, the cyber attack simulations conducted by coreLang, and the results from the random guessers. From that table, we can observe that while the human experts outperform the simulation in the smallest experiment, Experiment 1, in the rest of the experiments the human experts are inferior to the simulations.

Table 4
Average and longest time spent by human experts and time required by simulation for each experiment.

	Human Domain Experts		Simulation
Exp.	Average Time Spent	Longest Time Spent	Time Required
1	25 minutes	30 minutes	4 seconds
2	40 minutes	45 minutes	1 minute
3	38 minutes	45 minutes	≈1.5 minutes
4	204 minutes	296 minutes	≈3 minutes

Fig. 7 illustrates the root squared errors of the human experts and the random guessers per question answered in Experiment 4, compared to the ground truth. The mean is represented by an 'x' mark on each bar, while the horizontal line within the box represents the median value. From this figure, the simulation results are not included since they would only represent a single point for each question, adding no additional value to the figure. Additionally, the inconsistency in the errors of the random guessers is due to the generation of different random values for each question by each random guesser.

Fig. 8 presents a comparison of the average RSE of the experts' assessments with those of the random guessers and the simulations for each question in each of the Experiments 2-4.

Furthermore, Figs. 9–11 showcase radar charts displaying each expert's error against the ground truth for the questions in Experiments 2–4. These radar charts also include a comparison of the errors to the simulation results (represented by a dashed pink line) and the errors of the random guessers (represented by a dotted yellow line). In this type of charts, the closer a line is to the external perimeter, the lower the error of the assessor's answer for that specific question. By analyzing the error value for each point, we can discern whether an expert made a minor error in assessing the relative difficulty of the targeted attack goal – resulting in a low error value – or gravely misjudged the easiest attack goal to execute, leading to a high error value.

It is worth noting that no radar charts are presented for Experiment 1 due to the limited number of questions posed to the experts; only two out of the five questions pertained to reachable attack goals.

Table 5
Average root mean square error (RMSE) of assessments against the ground truth.

Assessor	Simulation (coreLang)	Human Domain Experts	Random Guessers
Experiments 1–4	3.9	4.5	6.3

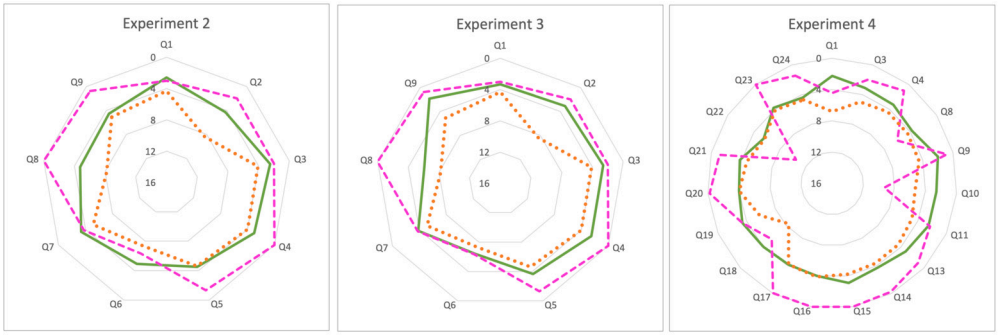


Fig. 8. Experts' average root squared errors plotted (solid green line) and compared with the simulation's error (dashed pink line) and the random guessers (dotted yellow line) for Experiments 2–4. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

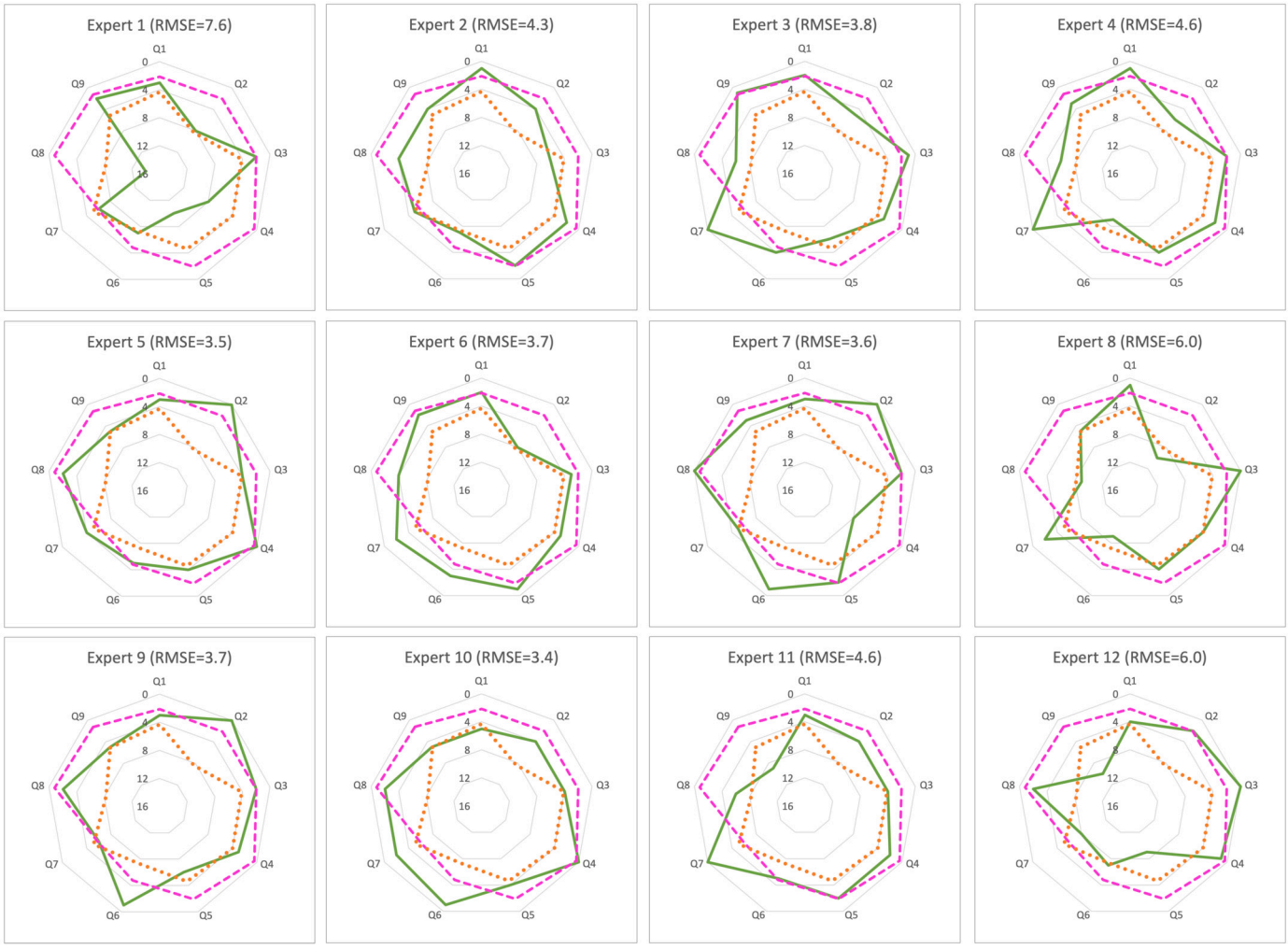


Fig. 9. Experts' root squared errors plotted individually and compared with the simulation's error (dashed line) and the random guessers (dotted line) for Experiment 2.

6. Analysis and comparison of results

In this section, we will analyze the results by highlighting the most significant findings derived from the comparison of the results.

One notable conclusion from Table 5 is that, on average, the attack simulation's assessments outperform the assessments of human experts. More specifically, this is true for experiments 2–4 but not for experiment 1 due to the lack of information in the model. However, it is

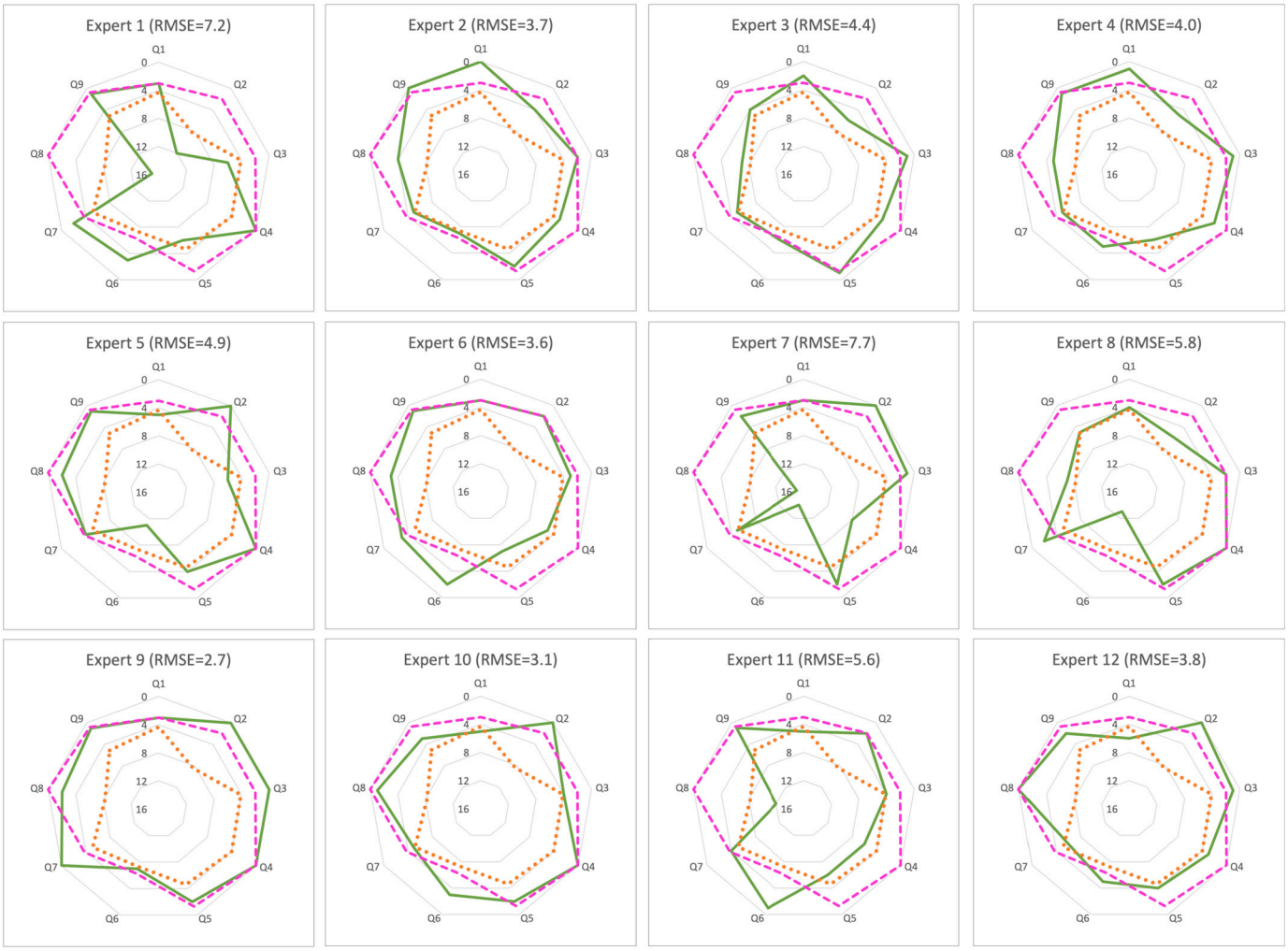


Fig. 10. Experts' root squared errors (solid line) plotted individually and compared with the simulation's error (dashed line) and the random guessers (dotted line) for Experiment 3.

worth mentioning that certain human experts did achieve better assessments for specific questions in the experiments. Hence, it appears that simulations could be beneficial for conducting a substantial number of assessments, while human experts might find value in concentrating on their specific areas of expertise to validate and complement the simulation results. However, further investigation and validation would be necessary to solidify these observations.

Furthermore, the comparison against random guessers reveals that both the simulations and the human experts consistently outperform the random guessers. The average RMSE of the random guessers across all experiments was 6.3, indicating that the simulations and the human experts consistently achieved significantly better results on average compared to random guessing.

To further evaluate that conclusion based on the results we have gathered, a post processing and analysis approach was used where we calculated the difference between the simulation's assessment error and the human experts' assessment error for each one of the questions asked to them in Experiments 2–4. Those values allow us to first identify how often the simulation's assessments outperform the human experts' assessments and in addition allow us to statistically analyze them by calculating the Confidence Interval (CI) and performing a statistical t-test. In Fig. 12 the values calculated are presented in a histogram. From that figure we observe that for almost over the 70% of the cases, the simulation outperforms or performs equally to the experts, while the human experts only outperform simulation 30% of the time.

Subsequently, we computed the 95% confidence interval for the mean difference in performance between humans and simulations. This interval furnishes us with a range in which we can be 95% certain that the true population mean resides. This calculation yielded an interval of $(-1.55, -0.86)$. Given that both ends of this interval are negative, it implies that, on average, simulations tend to outperform human experts.

Lastly, we performed a null hypothesis t-test as follows:

$H_0 : \mu = 0$ (there is no difference in performance between humans and simulations on average)

$H_a : \mu \neq 0$ (there is a significant difference in performance between humans and simulations)

Where:

H_0 is the null hypothesis

H_a is the alternative hypothesis

and μ represents the population mean difference in performance

And the results of the t-test were calculated as:

t-statistic: -6.92

p-value: 1.57×10^{-11}

Given that the p-value is much smaller than the significance level (which is $\alpha = 0.05$ in our case since the distribution of our dataset resembles a normal distribution), we reject the null hypothesis and we can conclude that there is a statistically significant difference in perfor-

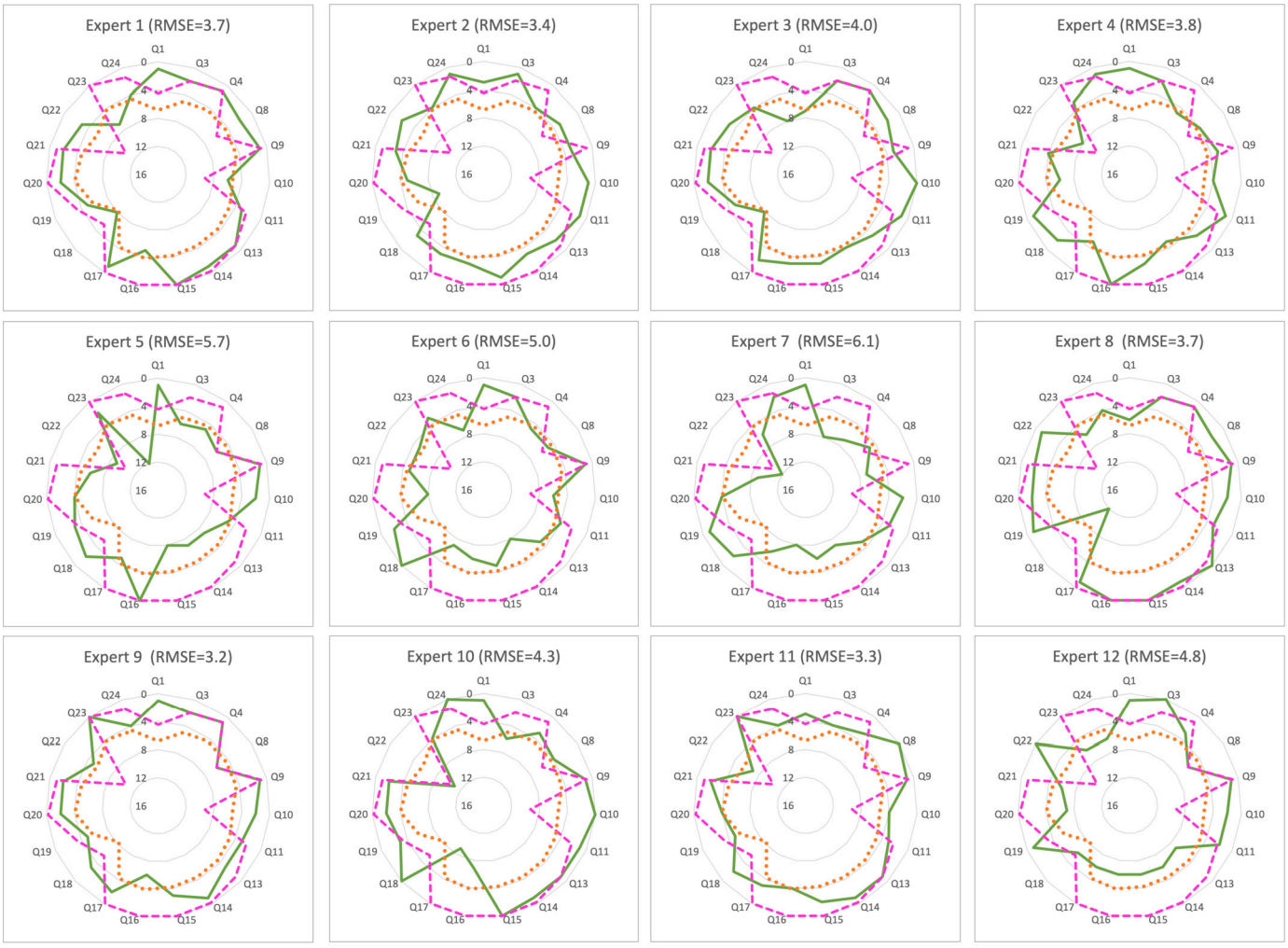


Fig. 11. Experts' root squared errors (solid line) plotted individually and compared with the simulation's error (dashed line) and the random guessers (dotted line) for Experiment 4.

mance between humans and simulations. The negative *t*-statistic further confirms that humans, on average, perform worse than simulations.

Subsequently, we performed a similar statistical analysis for the difference between the human experts' assessment error and the random guessers' assessment error. This analysis yielded a confidence interval of $(-1.69, -1.42)$, and in the t-test, the *t*-statistic was -10.18 with a corresponding *p*-value of 5.4×10^{-22}. These results once again indicate a statistically significant divergence in performance between humans and random guessers, favoring the expertise of human assessors.

Analysis of the results revealed that the dimension of time held particular significance for human expert assessors, given that cyber attack simulation results and assessments from random guessers were produced almost instantly. However, when accounting for modeling time (i.e., the time required to create the model of the infrastructure), cyber attack simulations still prove to be superior. This is because the modeling time is shorter than the time needed for human experts to assess a complex infrastructure.

Comparing the assessments of the human experts with each other reveals significant differences in their evaluations. Additionally, there is no consistent top performer among the human experts across all experiments. Instead, in each experiment, a different human expert performed best, achieving the lowest error against the ground truth. In Experiment 2, none of the experts achieved a lower error than the simulation, indicating that the simulation's assessment outperformed all the human experts in that particular experiment. In Experiment 3, only one human expert managed to achieve a lower error than the simulation, suggesting that the experts' assessment was closer to the ground truth than the simulation's assessment. However, in Experiment 4, half of the human expert participants were able to outperform the attack simulation's assessment, with three of them achieving a lower RMSE.

A closer examination of individual human experts reveals that Expert 10 was the top performer in Experiment 2, while Expert 9 excelled in both Experiment 3 and Experiment 4, achieving an RMSE of 2.7 and 3.2 respectively.

Additionally, in the evaluation of human experts' ability to identify and differentiate the unreachable attack goals and honeypots in Experiment 4, they achieved an average accuracy of approximately 50%.

Regarding Experiment 1, the simulation's assessment could not generate accurate results due to the lack of information in the model. Inadequate modeling of the infrastructure prevented the simulation from calculating a successful attack path toward the desired attack goals. Specifically, the model lacked information about vulnerable credentials and vulnerabilities, limiting the attacker to only having network reachability on the host. Consequently, all attack goals were considered of equal difficulty, resulting in a high error in the assessment. Interestingly, the human experts outperformed the simulation in Experiment 1 by leveraging their experiences utilizing their expertise and experiences to compensate for the missing information. This highlights that there is a threshold of information required in a model for the simulation's assessments to be valuable.

Ultimately, revisiting our research objective of falsifying coreLang, we conclude that this attempt was unsuccessful. Given the level of mod-

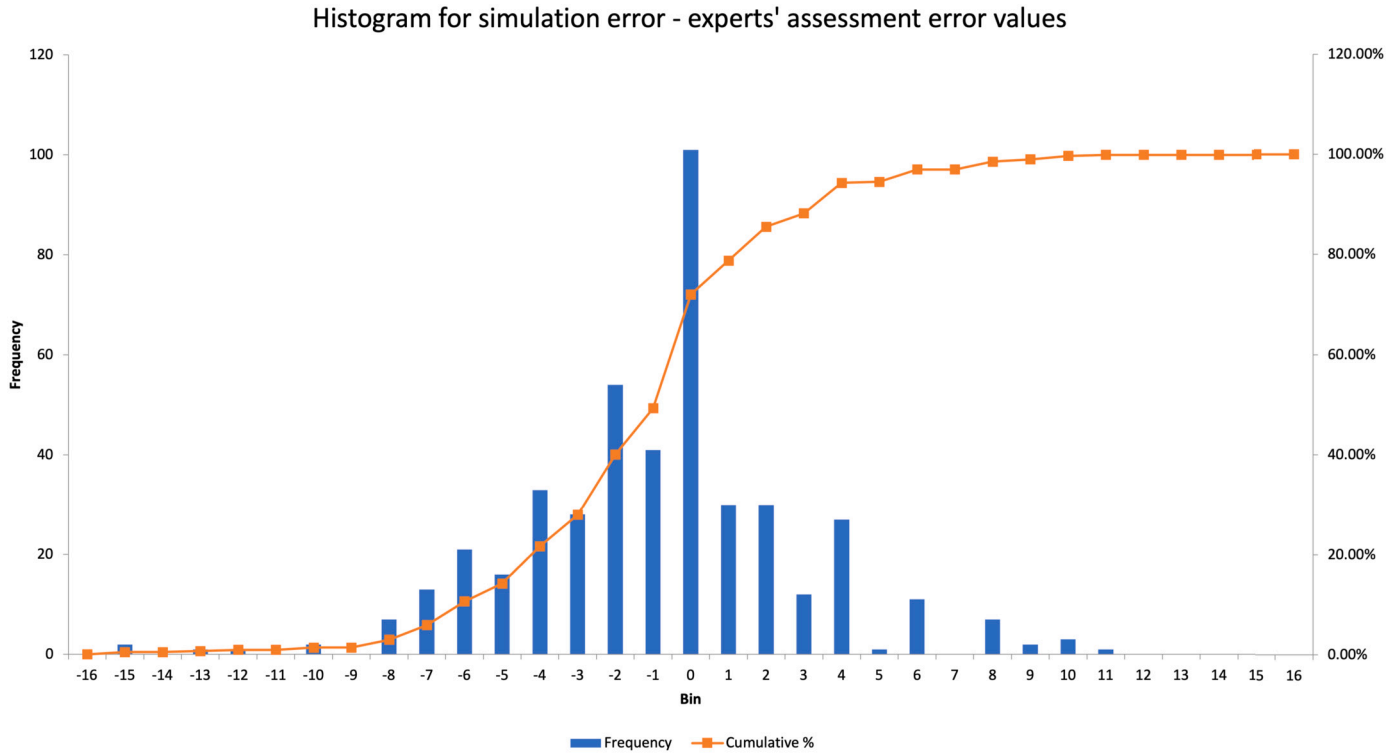


Fig. 12. Histogram of the simulation and human expert error difference values.

eling abstraction inherent in coreLang, and the fact that its assessments proved to be more reliable and credible than initially anticipated, we can assert that our study has corroborated its validity within the confines of our research parameters.

7. Discussion

Threat modeling is a promising approach for assessing the cybersecurity of large ICT infrastructures, and one way to achieve this is by using domain-specific modeling languages to create models and perform cybersecurity assessments. In this study, we proposed and applied a method to corroborate coreLang, a specific domain-specific language for threat modeling in the ICT domain, through an attempt at falsification.

Firstly, we assume that had the domain-specific language for this study been tailored specifically to the domain, since coreLang was created to be a fit for modeling any ICT infrastructure and therefore has a high level of modeling abstraction, the simulation results would have been more precise and detailed. Adapting the languages to align with the domain's unique needs could enhance performance and more aptly assess the security of cyber infrastructures.

Furthermore, a potential concern for this study might be the limited time allotted to the human experts. It is conceivable that given more time, these experts could provide markedly improved assessments of the infrastructure's cybersecurity. However, the time constraint was set based on our experts' maximum availability, and experiments were accordingly designed to fit within that timeframe. Moreover, after each experimental session, we inquired about our experts' confidence in their responses. On average, they expressed moderate to high confidence in their answers for all experiments, with the exception of Experiment 1, which was tightly time constrained by design.

Furthermore, a plausible objection to this study might arise due to our role as the developers of MAL and coreLang. It could be raised as a concern that our involvement could potentially introduce bias into the study's design or the modeling process. For instance, skepticism might arise about the experiment's fairness or whether the infrastructure's model created by coreLang was deliberately tailored to yield favorable simulation results. It is important to remind that in order to mitigate potential bias, during the experimental design phase, we had not yet created the coreLang models of the infrastructure. Having a coreLang developer involved in the modeling process ensured the language was used as intended. Ideally, the language would be entirely intuitive, preventing modelers from inadvertently creating models that do not mirror reality. While this usability aspect is not addressed in our study, the coreLang developer's participation helped mitigate the occasional lack of intuitiveness when modeling with coreLang.

In addition, it is important to address aspects of validity, reliability, and generality in the context of this study. One potential factor affecting reliability is the selection of human domain experts, as the study outcomes are closely tied to their participation. To mitigate this potential concern, we have involved a substantial number of experts in our study. However, it is worth noting that if more time had been available, then we could have included an even larger group of experts, that would further enhance the reliability of our results.

Another crucial aspect of reliability to consider is our reliance on a single, experimental cyber range for our assessed infrastructure, which contained a limited number of vulnerabilities. It's possible that utilizing a different cyber range might have yielded varied results. Nonetheless, we believe that the experimental cyber range employed in this study is quite realistic and includes a fairly representative set of vulnerabilities that our assessors were tasked to security assess.

In terms of the simulation results, both validity and reliability are ensured, given that the simulation's assessments would remain consistent for the same modeled infrastructure and the same attack goals.

Regarding the validity of our student data, it is important to clarify that while the human domain experts are considered cybersecurity professionals, and have the expertise to understand how a penetration tester would act, the students who represent the ground truth are not professional penetration testers. They did however have in their disposal a lot of guidance by the on-demand hints and other resources provided to them during the course, which made them able to successfully capture all the flags. The data collected from them are used to

construct the ground truth measures of relative attack path difficulty. Our interest in the relative difficulty of attack paths makes that data more relevant to us. For that reason, it is not necessary that the students are equally proficient as professional hackers, but only that both groups have similar perceptions of relative difficulty. So, for example, even though professional attackers might complete the binary exploitation in shorter time than students, both would likely find that more difficult than brute forcing an FTP server with a very simple numeric password.

Another potential validity concern is the way we asked questions to our assessors. While we chose pairwise comparison questions, an alternative approach could involve asking assessors to provide absolute time estimates for each attack goal. However, we opted for pairwise comparisons because coreLang, as an abstract modeling language, does not provide high-resolution time estimates for individual attack steps, and because evaluating the relative difficulty of various attacks holds greater significance for practical decision-making, as it necessitates prioritizing security concerns that require attention.

Touching on our study's generality, we posit that altering the infrastructure to be assessed by the human experts and by coreLang would not substantially impact the results because coreLang can adequately model and simulate attacks on any ICT infrastructure.

Additionally, the selection of our human experts is a topic for discussion and a weakness of our study. As mentioned earlier, the experts who responded to our time-bounded open call for participation were qualitatively assessed by us as researchers, without using a formal assessment method. This approach was driven by two primary reasons. First, we aimed to compare the language against an average cybersecurity professional, positing this as a benchmark to showcase where the language begins to offer benefits. Second, establishing a reference group of higher expertise would have presented logistical challenges, both in appropriately qualifying individuals and in recruiting participants willing to dedicate time and effort to the experiment. Another point we can make is that the expertise required from our participants was not proficiency in penetration testing, but rather the ability to conduct a high-level security analysis. For example, an expert's lack of skill in executing an SQL injection attack does not deem them less suitable for our study.

Finally, revisiting our hypothesis that analyzing an infrastructure of increased complexity would significantly impede the human experts' assessment process, we can conclude that this is still an open question to us since our results were inconclusive towards that direction. To understand exactly how much and under what circumstances the human experts perform significantly worse than simulations, would require another experimental setup.

8. Conclusion

The most important conclusion drawn from this study is that the simulation results generated using coreLang outperform on average the assessments performed by human domain experts.

Another noteworthy conclusion is that a small number of human experts can still handle the challenging task of assessing a large and initially unknown infrastructure within a limited timeframe and individually achieve a higher level of accuracy in their assessments compared to the rest of the group. When given more time, some of the human experts are able to provide better assessments that are comparable or slightly better than the simulation's assessments.

Another conclusion is that the efficiency of the simulation's assessments, that are being generated almost instantly, cannot be matched by the human domain experts. Furthermore, both human domain experts as well as simulations are significantly better than random guessers in their assessments.

As a closing remark, it is worth mentioning that while coreLang has been validated and shown to be, on average, better than human domain experts, there is still room for further improvements to make its assessments even more realistic. Nevertheless, this study demonstrates that progress is being made in the right direction.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used OpenAI's language model, ChatGPT in order to proofread and optimize the text of this work. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

Allodi, L., Massacci, F., Williams, J., 2022. The work-averse cyberattacker model: theory and evidence from two million attack signatures. Risk Anal. 42 (8), 1623–1642.

Bilge, L., Dumitraş, T., 2012. Before we knew it: an empirical study of zero-day attacks in the real world. In: CCS '12. Association for Computing Machinery, New York, NY, USA, pp. 833–844 [Online].

Connolly, L. Yuryna, Wall, D.S., Lang, M., Oddson, B., 2020. An empirical study of ransomware attacks on organizations: an assessment of severity and salient factors affecting vulnerability. J. Cybersecurity 6 (1), tyaa023. https://doi.org/10.1093/cybsec/tyaa023 [Online].

Dahbul, R.N., Lim, C., Purnama, J., 2017. Enhancing honeypot deception capability through network service fingerprinting. J. Phys. Conf. Ser. 801 (1), 012057. https://doi.org/10.1088/1742-6596/801/1/012057 [Online].

Ekstedt, M., Johnson, P., Lagerström, R., Gorton, D., Nydrén, J., Shahzad, K., 2015. Securi CAD by Foreseeti: a CAD tool for enterprise cyber security management. In: 2015 IEEE 19th International Enterprise Distributed Object Computing Workshop, pp. 152–155.

Feigenbaum, E., 2003. Some challenges and grand challenges for computational intelligence. J. ACM 50, 32–40.

Ferro, L.S., Marrella, A., Catarci, T., 2021. A human factor approach to threat modeling. In: Moallem, A. (Ed.), HCI for Cybersecurity, Privacy and Trust. Springer International Publishing, Cham, pp. 139–157.

Hacks, S., Katsikeas, S., Ling, E. Rencelj, Xiong, W., Pfeiffer, J., Wortmann, A., 2022. Towards a systematic method for developing meta attack language instances. In: Enterprise, Business-Process and Information Systems Modeling. Springer International Publishing, Cham, pp. 139–154.

Johnson, P., Lagerström, R., Ekstedt, M., 2018. A Meta language for threat modeling and attack simulations. In: Proceedings of the 13th International Conference on Availability, Reliability and Security. ARES 2018, New York, NY, USA.

Katsikeas, S., Hacks, S., Johnson, P., Ekstedt, M., Lagerström, R., Jacobsson, J., Wällstedt, M., Eliasson, P., 2020. An attack simulation language for the IT domain. In: Eades III, H., Gadyatskaya, O. (Eds.), Graphical Models for Security – 7th International Workshop. GraMSec 2020, Boston, MA, USA, June 22, 2020 Revised Selected Papers. In: Lecture Notes in Computer Science, vol. 12419. Springer, pp. 67–86 [Online].

Katsikeas, S., Johnsson, P., Hacks, S., Lagerström, R., 2022. VehicleLang: a probabilistic modeling and simulation language for modern vehicle it infrastructures. Comput. Secur. 117, 102705 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404822001031.

Lallie, H.S., Debattista, K., Bal, J., 2018. An empirical evaluation of the effectiveness of attack graphs and fault trees in cyber-attack perception. IEEE Trans. Inf. Forensics Secur. 13 (5), 1110–1122.

Lallie, H.S., Debattista, K., Bal, J., 2020. A review of attack graph and attack tree visual syntax in cyber security. Comput. Sci. Rev. 35, 100219 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1574013719300772.

MITRE, 2024a. ATT&CK Matrix for Enterprise [Online]. Available: https://attack.mitre.org/.

MITRE, 2024b. The common vulnerabilities and exposures (CVE) program [Online]. Available: https://cve.mitre.org/.

MITRE, 2024c. Common weakness enumeration (CWE) [Online]. Available: https://cwe.mitre.org/.

Nobles, C., 2018. Botching human factors in cybersecurity in business organizations. HOLISTICA J. Bus. Public Adm. 9 (3), 71–88.

Nydrén, J., 2023. A better way to stay ahead of attacks: security command center adds attack path simulation [Online]. Available: https://cloud.google.com/blog/products/identity-security/security-command-center-adds-attack-path-simulation-to-stay-ahead-of-cyber-risks/.

Popper, K., 2005. The Logic of Scientific Discovery. Routledge.

Saaty, T.L., 1984. The Analytic Hierarchy Process: Decision Making in Complex Environments. Springer US, Boston, MA, pp. 285–308 [Online].

Schmidhuber, J., Hochreiter, S., Bengio, Y., 2001. Evaluating benchmark problems by random guessing. In: A Field Guide to Dynamical Recurrent Networks, pp. 231–235.

Tayouri, D., Baum, N., Shabtai, A., Puzis, R., 2023. A survey of MulVAL extensions and their attack scenarios coverage. IEEE Access 11, 974–27 991. https://doi.org/10.1109/ACCESS.2023.3257721 [Online].

Xiong, W., Lagerström, R., 2019. Threat modeling – a systematic literature review. Comput. Secur. 84, 53–69 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404818307478.

Yskout, K., Heyman, T., Van Landuyt, D., Sion, L., Wuyts, K., Joosen, W., 2020. Threat modeling: from infancy to maturity. In: 2020 IEEE/ACM 42nd International Conference on Software Engineering: New Ideas and Emerging Results. ICSE-NIER, pp. 9–12.



Sotirios Katsikeas is a PhD candidate at the School of Electrical and Computer Engineering at the Royal Institute of Technology (KTH) in Stockholm, Sweden. His main topic of research is domain specific languages for cyber attack modeling and simulations. He received his Diploma (5-year program) in Electrical and Computer Engineering from the Technical University of Crete in Greece and his MSc from the Royal Institute of Technology (KTH) in 2018.





Engla Rencelj Ling has been a Ph.D. student in the Networking and Systems Engineering division at the Royal Institute of Technology (KTH) in Stockholm, Sweden since January 2019. Her research topic is threat modelling for digitalized power systems. She received her M.Sc. in Computer and Information Engineering from Uppsala University in 2015. Between 2015 and 2019 she worked as a Systems Engineer at Cisco.

Pontus Johnsson is a professor at the Royal Institute of Technology (KTH) in Stockholm, Sweden. His research interests mainly lie in the area of cyber security and the analysis of architectural models of computer networks – in particular simulating cyber attacks on such networks. Pontus supervises a number of PhD students and holds courses on Ethical Hacking. He is the director of the Center for Cyber Defense and Information Security at KTH. He received his MSc from the Lund Institute of Technology in 1997 and his PhD and Docent titles from the Royal Institute of Technology in 2002 and 2007. He was appointed professor in 2009.



Mathias Ekstedt is a Professor of Industrial Information and Control Systems at KTH Royal Institute of Technology in Stockholm, Sweden from where he also received his Ph.D. in 2004. His research interests include the intersection of cybersecurity and software systems architecture modeling and analysis. Mathias is the co-founder and director of KTH's Master programme in Cybersecurity.